머신러닝 첫걸음

역사와 로드맵 · 문제 정식화와 세 갈래 분류 · 파이프라인과 사전지식 리뷰

Machine Learning 1 · 1주차

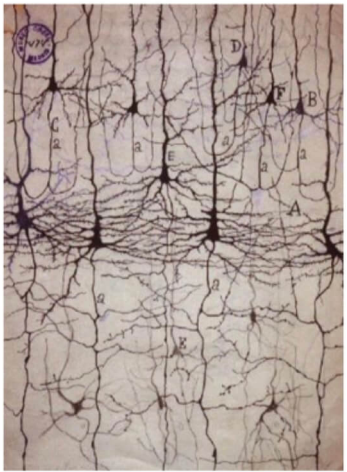
한국과학영재학교 (KSA)

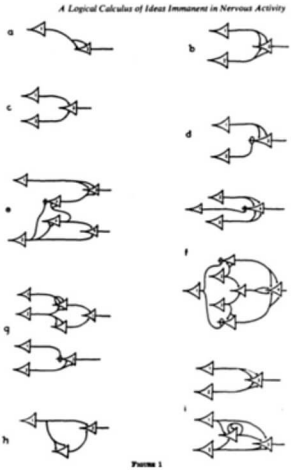
Week 1

목차

- **역사**: 고전 ML 계보, 퍼셉트론 $\rightarrow$ LLM 연표, 두 번의 “AI 겨울”
- **학기 로드맵**: Block A(고전 ML) · Block B(신경망 $\rightarrow$ 생성모델)
- **문제 정식화**: 입력 x · 출력 y · 가설 h, 정답 라벨은 사람이 정한다
- **세 갈래 분류**: **지도 / 비지도 / 강화**, 판별 vs 생성, 회귀 vs 분류
- **파이프라인과 수학**: 수집 $\rightarrow$ 평가, 데이터 누수, **내적 · 연쇄법칙 · 베이즈**

1.1 고전 ML과 신경망의 역사, 이번 학기 로드맵

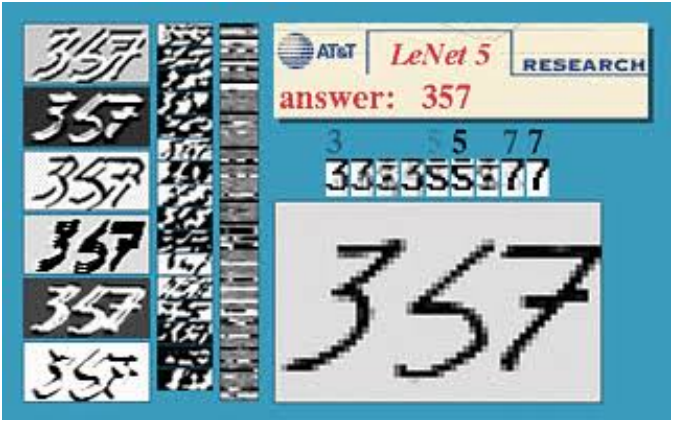




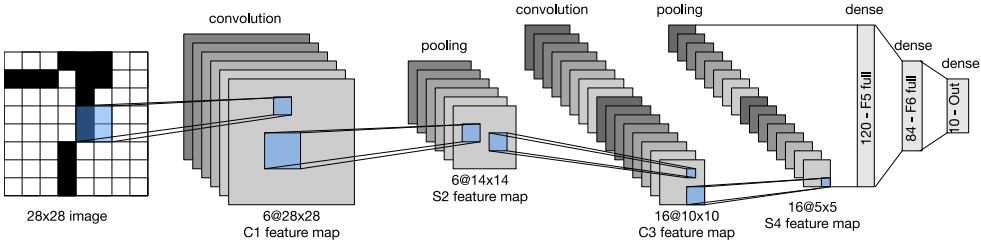
좌: Ramon y Cajal의 신경 해부도 (1890s)
우: McCulloch & Pitts의 로직 회로도 — 기본 신경원 단위로 AND, OR, NOT 등 모든 로직 구현 (1943)



Yann LeCun — 2018 Turing Laureate



LeNet(CNN) — MNIST Demo (1998)



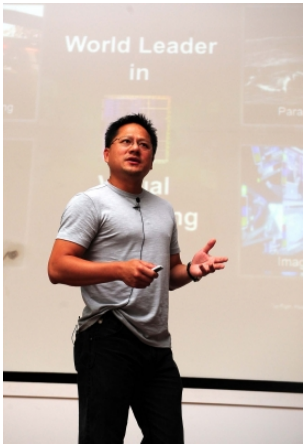
LeNet-5 Architecture (LeCun et al., 1998)



Fei-Fei Li (Stanford)
ImageNet 창시자



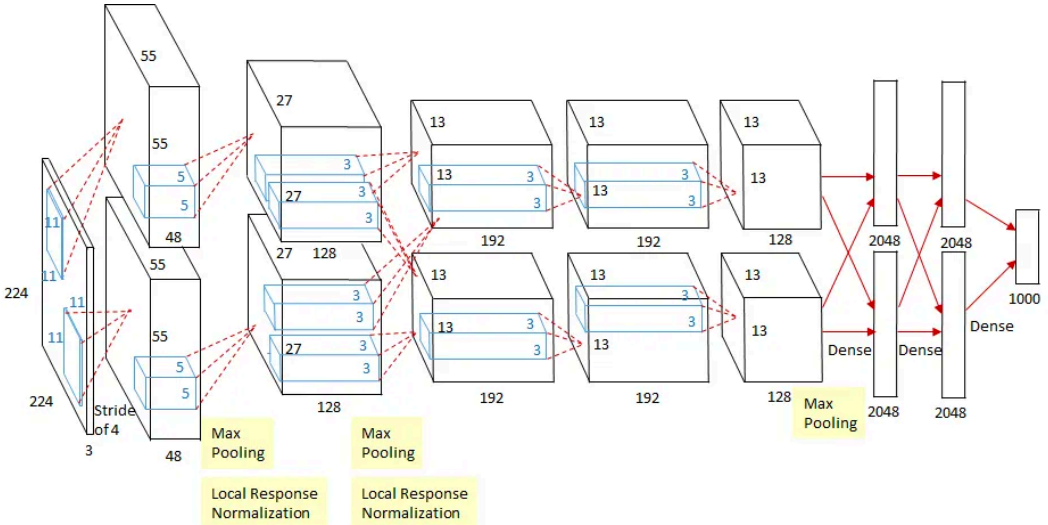
ImageNet (CVPR, 2009) — 총 **1,400만 장** (22K 카테고리)
ILSVRC Challenge (2010~, 1000 categories, 약 128만 장)



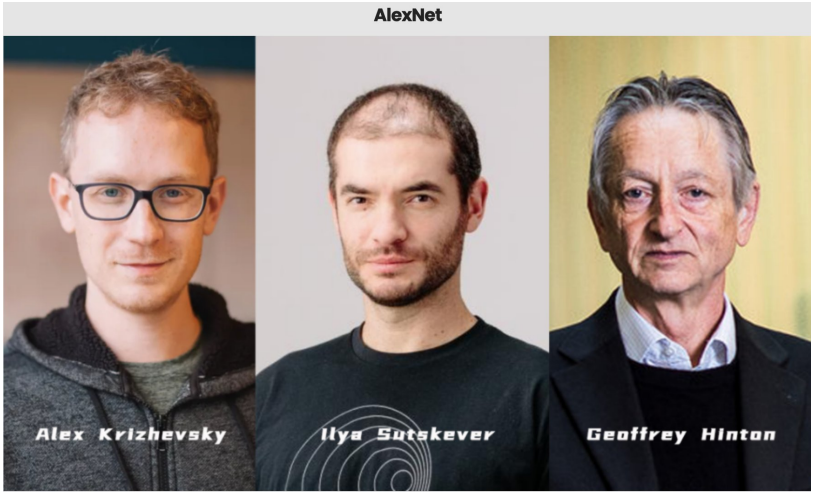
젠슨 황, 서울대 강연 (2008)
당시 엔비디아 주가 약 430원
(AlexNet 4년 전)



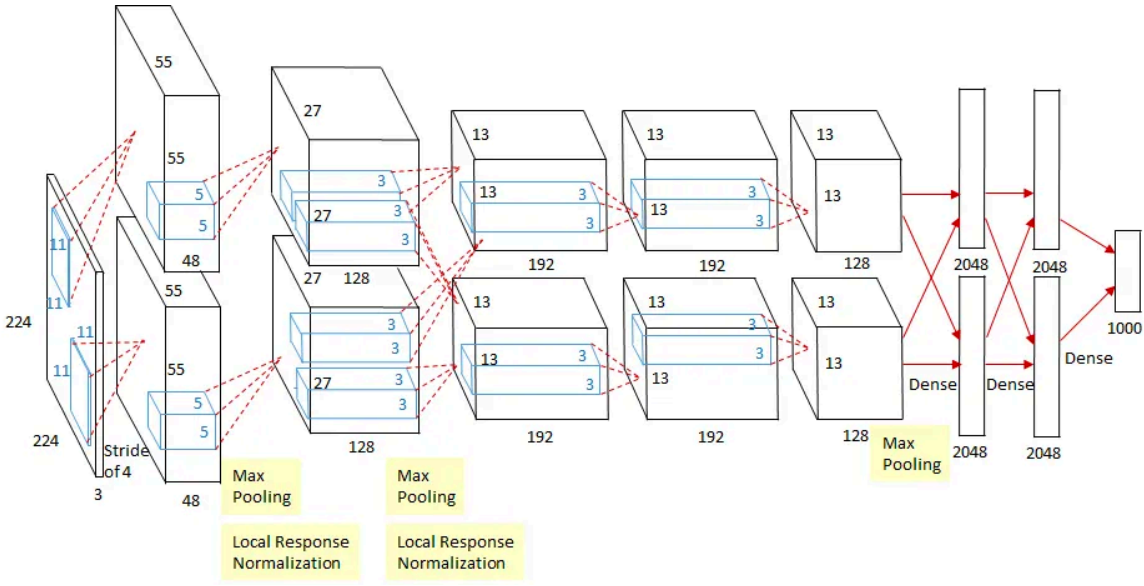
NVIDIA GeForce GTX 580
AlexNet(2012)이 이 GPU 두 장으로 5~6일간 학습



AlexNet (2012, Krizhevsky, Sutskever & Hinton, University of Toronto)



Krizhevsky: ImageNet 우승 • Sutskever: OpenAI, ChatGPT • Hinton: 2024 Nobel Prize


55
55
48
55
48
27
27
128
27
128
13
13
13
192
13
13
13
192
13
13
128
2048
2048
2048
2048
1000
Dense
Dense
Dense
224
224
3
11
11
11
11
5
5
5
5
3
3
3
3
3
3
3
3
Stride of 4
Max Pooling
Local Response Normalization
Max Pooling
Local Response Normalization
Max Pooling

이 과목이 다루는 질문

2012년 ImageNet 대회에서 **AlexNet** 이 등장하기 전까지, “이 사진에 고양이가 있는가”를 알려주는 방법은 **사람이 직접 특징을 설계**하는 것이었다 — 가장자리 검출기, 색 히스토그램, 특정 모양의 필터를 손으로 조합해 “고양이스러움”을 수치로 정의하려 했다.

AlexNet은 그런 손설계 없이, 수백만 장의 사진과 정답 라벨만 보고 **스스로 무엇을 봐야 하는지 학습**해서 기존 방법들을 압도적인 차이로 이겼다.

규칙을 사람이 짜는 대신, 정답이 있는 데이터로부터
기계가 직접 규칙을 찾아내게 하려면 어떻게 해야 할까?

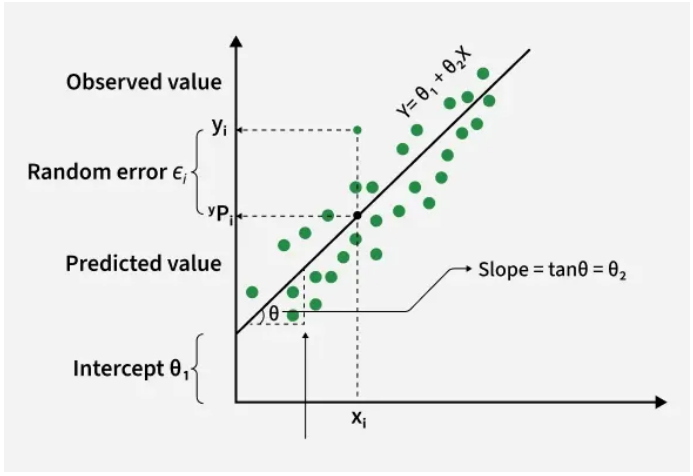

IMAGENET

두 갈래의 역사 — 고전 ML

고전 ML — 통계적 학습

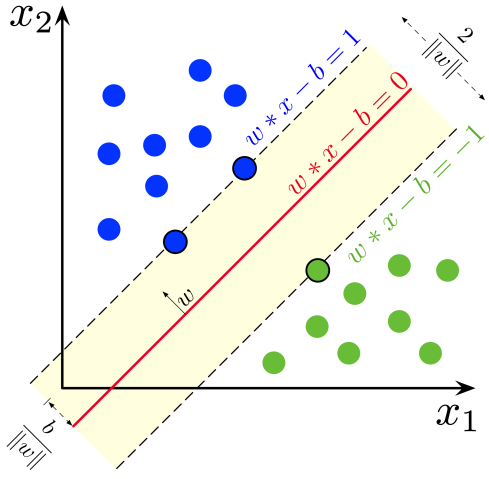
연도	사건
1805	최소제곱법(르장드르·가우스) — 회귀의 기원
1936	피셔의 선형판별분석(LDA)
1948	섀넌의 정보이론(엔트로피) — 결정트리 분할 기준의 토대
1951	최근접이웃 규칙(Fix&Hodges) → kNN
1986	결정트리 ID3(Quinlan)
1995	SVM(Cortes & Vapnik)
2001	랜덤포레스트(Breiman)

그림으로: 선형회귀 (1805, 최소제곱법)



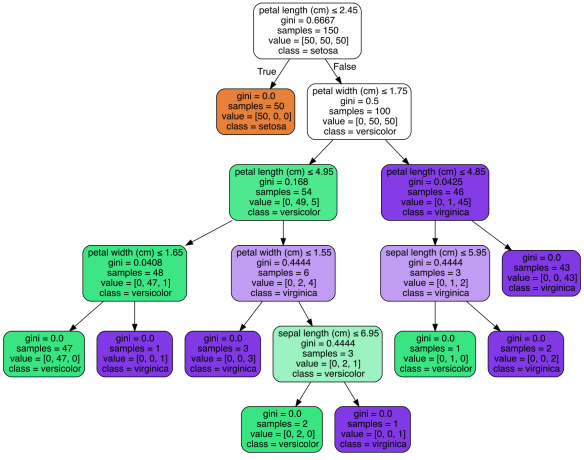
관측값 y_i 와 직선의 예측값 사이의 간격(random error ϵ_i)을 최소화하는 절편 θ_1과 기울기 θ_2를 찾는 문제 — Week 2에서 다시 만난다.

그림으로: SVM (1995, Cortes & Vapnik)



두 클래스를 가르는 무수히 많은 직선 중, 가장 가까운 점(서포트 벡터, 굵은 테두리)까지의 여백(margin, $2/\|w\|$)을 **최대로** 하는 하나를 고른다 — Week 5에서 다시 만난다.

그림으로: 결정트리 (1986, ID3)



붓꽃(iris) 데이터를 “꽃잎 길이 ≤ 2.45?”처럼 **하나의 질문**으로 계속 쪼개, 각 가지 끝(잎)에서 하나의 클래스로 수렴 — Week 7에서 다시 만난다.

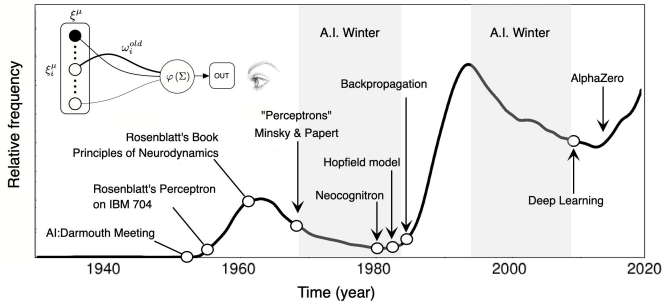
두 갈래의 역사 — 신경망

- 흥미로운 점: 역사가 직선으로 발전한 게 아니라 **침체기("AI 겨울")를 두 번이나 거쳤다**는 것.

신경망 — 퍼셉트론 → LLM

연도	사건
1943	맥컬록-피터즈 신경원 모형
1950s	퍼셉트론(단층 신경망) 등장
1969	XOR 한계 증명 → **첫 AI 겨울**
1986	역전파 재발견 → 다층 학습 가능
1990s∼2000s	SVM·커널기법에 밀려 **두 번째 AI 겨울**
1998	(겨울 중에도) LeCun의 CNN(LeNet)
2012	AlexNet — **딥러닝 시대의 시작**
2013	word2vec·VAE
2016	AlphaGo(이세돌) — 강화학습의 상징적 승리
2017	Transformer
2022	ChatGPT(사전학습+RLHF)
2025	멀티모달 LLM 대중화

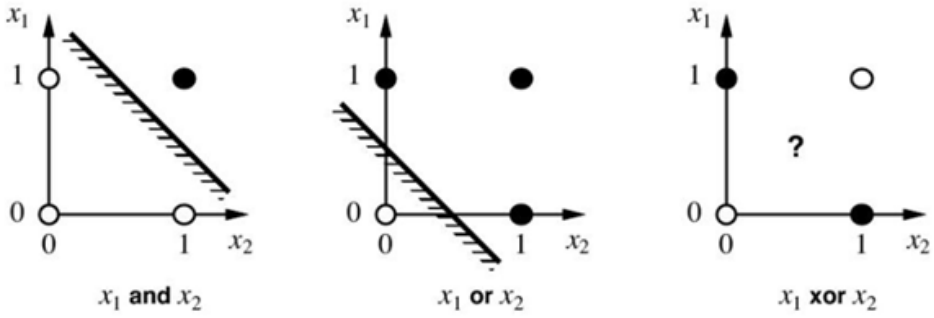
연표를 "연도 축" 위에 그리면



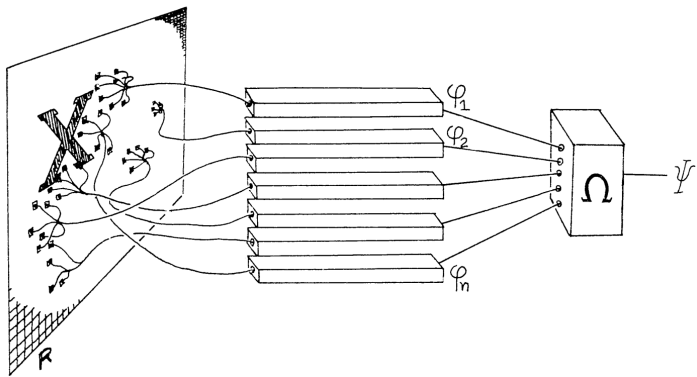
출처: AI voor Iedereen (KU Leuven), aivooriedereen.cs.kuleuven.be

- **첫 번째** 음영(1969→1986, **17년**): XOR 한계 증명 → 역전파로 해소.
- **두 번째** 음영(1990년대 중반→2000년대 후반): 딥러닝 재부상 직전까지 신경망이 다시 밀려난 시기.
- **2012년 이후**(AlexNet, AlphaZero)엔 간격이 짧아짐.

XOR — 직선 한 줄로 나눌 수 있을까?



- **AND**와 **OR**: 검은 점과 흰 점을 직선 하나로 가를 수 있다.
- **XOR**(“둘 중 정확히 하나만 1”): 검은 점 두 개가 대각선 모서리에 놓여, 직선 하나로는 가를 수 없다.
- 1969년 민스키·페퍼트가 이 한계를 증명했고, 이것이 첫 번째 음영의 출발점이다.



Rosenblatt의 퍼셉트론 구조 (1958) — 망막(R) $\rightarrow$ 연합부 $(\varphi_1, \ldots, \varphi_n)$ $\rightarrow$ 반응부(Ω)

1969 — 첫 겨울의 시작

- 민스키·페퍼트가 단층 퍼셉트론은 **XOR**(배타적 논리합, “둘 중 정확히 하나만 참일 때만 참”)조차 표현할 수 없음을 **수학적으로 증명**.
- “아직 알고리즘이 부족하다”가 아니라, “이 구조로는 **원리적으로 불가능**하다”는 훨씬 강한 주장.
- 결과: 연구비가 끊기고 연구자 수가 급감 — **“첫 AI 겨울”**이 10년 넘게 이어짐.

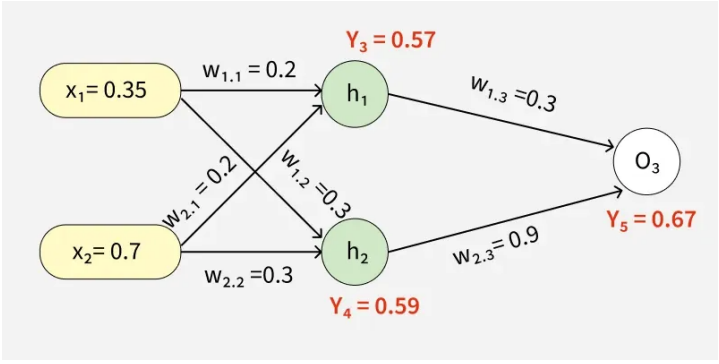
흥미로운 점

답(층을 여러 개 쌓으면 XOR도 표현 가능)은 **이미 알려진 수학** 안에 있었지만, “그 여러 층을 **어떻게 학습**시킬 것인가”라는 **계산적 문제**가 안 풀려서 겨울이 그토록 길어졌다.

Minsky, M. & Papert, S. (1969). Perceptrons: An Introduction to Computational Geometry. MIT Press.

1986 — 역전파가 겨울을 끝내다

- 러멜하트·힌튼·윌리엄스(1986)가 역전파를 (재)발견.
- 여러 층을 가진 신경망도 **경사하강법으로 실제로 학습**시킬 수 있다는 것이 드러나.
- 정확히 1969년의 한계(XOR)를 **실제로 풀어내는** 방법이었다.



Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323(6088), 533–536.

2012 — 딥러닝 시대의 개막

- AlexNet은 1986년의 역전파, 1998년의 CNN(LeNet)과 근본적으로 **다른 새 수학을 쓴 게 아니었다**.
- 결정적 차이: **GPU로 병렬 연산**하게 되면서 계산량이 감당 가능해졌고, 인터넷 덕분에 ImageNet 같은 **대규모 라벨링된 데이터셋**이 존재했다.

"좋은 아이디어가 있어도 데이터와 계산 자원이 받쳐주지 않으면 묻힐 수 있다."



Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez*† University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin*‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

†Work performed while at Google Brain.

‡Work performed while at Google Research.

Transformer 아키텍처 (Vaswani et al., “Attention Is All You Need,” 2017)

2022∼2023 — 정렬(alignment)과 프론티어 모델

- **InstructGPT (Ouyang et al., 2022)** — **RLHF**(인간 피드백 강화학습)로 언어모델을 "사람이 원하는 방식"에 맞춤 — **ChatGPT를 실제로 가능하게 한 기법**.
- **GPT-4 Technical Report (OpenAI, 2023)** — 대규모 멀티모달 모델. "**프론티어 모델**(frontier model)"이라는 용어가 이 시기부터 자리잡음.
- **DALL-E 2 (Ramesh et al., 2022)** — CLIP + 디퓨전으로 텍스트 → 이미지 생성. ChatGPT와는 **다른 갈래**(생성모델, Week 15)지만 같은 시대의 돌파구.

공통점: 근본적으로 새 수학이 아니라, Block A/B에서 배울 기존 아이디어(어텐션, 강화학습, 디퓨전)를 더 큰 규모로 조합한 것.

Ouyang, L. et al. (2022). Training language models to follow instructions with human feedback. NeurIPS.
OpenAI (2023). GPT-4 Technical Report. arXiv:2303.08774.

Ramesh, A. et al. (2022). Hierarchical Text-Conditional Image Generation with CLIP Latents. arXiv:2204.06125.

반복되는 패턴 — 돌파구의 정체

다수의 “돌파구”는 새 수학이 아니다

이미 있던 아이디어(경사하강법, 연쇄법칙, 확률론)를 **더 큰 데이터 · 더 많은 계산 자원** 위에서 **다시 조합**한 것이다.

- 1969 XOR 증명 $\rightarrow$ 17년 $\rightarrow$ 1986 역전파(바로 XOR의 해법).
- 1998 LeNet $\rightarrow$ 2012 AlexNet(같은 수학, GPU + ImageNet).

“왜 2012년이었을까” — 첫 번째 축: 계산

- AlexNet은 2012년 최신 GPU 두 장(NVIDIA GTX 580)으로 **약 5∼11일** 걸리던 학습.
- 같은 모델을 당시 CPU 클러스터로 돌리면 **월 단위**로 늘어나는 것이 일반적 추정.

연상해야 할 도구: 무어의 법칙

집적회로의 성능이 대략 2년마다 배가. 고정된 “한 학기 안에 끝나야 하는” 학습 시간 예산이 그대로라면, 1998년에는 LeNet 크기의 모델을 돌리는 데 거의 다 썼겠지만, 2012년에는 같은 예산 안에 **층과 폭이 훨씬 큰 모델**을 돌려볼 수 있었다.

신경망 연구에서 “구조”와 “계산”은 한 쪽만 커져서는 안 되는 한 쌍 (Week 9 역전파의 연산 비용을 손으로 셀 때 다시 등장).

“왜 2012년이었을까” — 두 번째 축: 데이터

- ImageNet: **약 140만 장**의 이미지에 **1000개 클래스** 라벨을 붙인 데이터셋 (2009년 공개).
- 중요한 점: 라벨 대부분이 **일반인이 직접 매긴 것** — 인터넷과 크라우드소싱(아마존 메카니컬 터크) 플랫폼이 없었다면, 100만 장 넘는 사진마다 “고양이”·“셔터”를 붙이는 비용이 **연구비를 압도**했을 것이다.

“140만 장”과 “2장 GPU”의 동시 출현

AlexNet의 승리는 두 축이 **동시에** 갖춰진 2012년에만 가능했던 것 — 어느 하나만 있었어도 그 해의 결과물은 달라졌다.

이번 학기 내내 등장하는 모든 모델이 이 “두 축” 위에 선다.

세 번째 교훈: 라벨의 형태가 모델의 형태를 결정한다

- 2012년의 패러다임 전환은 "**사진 + 범주 라벨**"이라는 **지도학습**의 가장 전형적인 형태 위에서 일어났다.
- 정답이 없는 데이터만 있을 때(비지도학습, Week 14∼15), 보상 신호만 있을 때(강화학습, ML2)에는 같은 GPU·데이터 축도 중요하지만,

"어떤 신호로 학습하는가"에 따라 최적의 모델 가족 자체가 달라진다.

- 1.2절에서 "**데이터의 형태가 문제를 결정한다**"고 할 때 뜻하는 바가 정확히 이것.
- 부수 효과: "우리 데이터가 작으니 신경망은 아직 이르다" 같은 실무 판단이 왜 흔한지도 자연스럽게 이해된다.

이번 학기 로드맵 — 전반부

Block A (Week 2–7): 고전 ML — 신경망 없이도 강력; 지금도 정형(tabular) 데이터에서 자주 이긴다.

2	회귀: 선형·로지스틱, 경사하강법, PR-AUC
3	생성 관점 분류: 나이브베이즈와 GDA
4	거리 기반·클러스터링: kNN, k-means, GMM/EM
5	SVM과 커널: 마진, 소프트 마진, 커널 트릭
6	정규화와 모델 선택: L1/L2, 교차검증
7	트리: 결정트리, 랜덤 포레스트, GBDT+SHAP
8	Block A 캡스톤 + 중간고사 대비

매 장 시작 전에 이 표를 펼쳐 “지금 어디서 무엇을 배우고, 어디로 이어지는지”를 확인하는 습관.

이번 학기 로드맵 — 후반부

Block B (Week 9–16): 신경망 → 생성모델

9	신경망 기초: 퍼셉트론 한계, 역전파, 정규화
10	CNN: 합성곱, 풀링, 탐지·분할·전이학습
11	시퀀스 모델: RNN, BPTT, LSTM/GRU
12	어텐션과 트랜스포머: QKV, Multi-Head
13	그래프 표현학습: Random Walk, Node2Vec
14	표현학습: PCA, word2vec, 임베딩 활용(RAG)
15	잠재변수 생성모델: VAE, GAN, Diffusion
16	Block B 캡스톤 + ML1 총정리

이번 학기 쓸 도구들 (수학 + 코드)

도구	이 학기에서 쓰이는 곳	언제 처음 등장
numpy	모든 실습의 기본. $w^\top x$, 행렬곱, 경사하강법 반복	Week 2
matplotlib	손실 곡선, 결정경계, t-SNE/PCA 시각화	Week 2
pandas	EDA, 결측치 처리, 그룹별 통계	Week 6 (1.3절 예습)
scikit-learn	분류·회귀·클러스터링 모델의 표준 인터페이스	Week 2
PyTorch	신경망·CNN·시퀀스 모델·VAE/GAN	Week 9
Hugging Face	LLM 프롬프팅, 사전학습 모델 활용	Week 17

이 목록을 "이번 학기에 **새로 배워야 할 것**"이 아니라, "이 학기 내내 **반복해서 손이 갈 것**"으로 읽는다 — 완벽히 외우는 것보다, **필요할 때 10초 만에 꺼내 쓸 수 있는 것**이 목표.

실습 노트북의 역할

- 각 절마다 붙는 실습 노트북(Colab 배지)은 **“개념을 손으로 확인”**에 집중.
- 이번 절의 노트북: 연표의 **“격차”를 직접 계산**, 2012년 이전의 AI 겨울을 **시각화**, 도구 목록의 라이브러리가 실제로 import 되는지 **확인**.

미리 강조

“어떤 모델을 쓰느냐”보다 **“도구가 손에 익었느냐”**가 이 학기 전반부의 진짜 성패를 가른다.

자주 하는 실수: “최신 모델”부터 잡기

- 가장 흔한 실패 패턴: Transformer(Week 12)·LLM(Week 17)이 “산업에서 쓰는 것”이라는 이유로 **Block A를 건너뛰고 Week 12로 직행**.
- 실제 경험: 작은 네트워크의 손실이 안 떨어지는데 (a) 데이터 스케일? (b) 학습률? (c) 구조 자체의 문제인지 구분 못 하고 “일단 층을 늘려보자”로 끝남.
- 한 달 뒤엔 **원인 모를 손실 곡선**만 남는다.

실전에서 막히는 진짜 이유

Week 6의 “train/val/test 분리”와 Week 2의 “손실함수를 손으로 한 번 계산” 같은 **기초 도구가 없기** 때문이다.

실수의 본질 — 그리고 로드맵이란

- 본질: “**최신 = 어렵다 = 배우면 실력이 된다**”는 착각.
- 실제로 최신 모델 논문의 한 줄 한 줄은 전부 Block A·B 기초 (경사하강법, 시그모이드, 교차엔트로피, 어텐션)의 **변형**.
- 기초가 없으면 논문을 “읽는” 게 아니라 “**눈으로 훑어보는**” 것에 그침.

- 반대로 Block A를 충실히 마치면 Transformer 논문을 읽을 때 “이 단락은 어텐션의 QKV를 다루는데, 12.1절의 QKV랑 같다”로 **바로 위치를 잡을 수 있다**.

로드맵은 “배울 것의 목록”이 아니라 “어디서 어떤 기초가 다시 쓰이는지”를 보여주는 지도.

자주 묻는 질문 2: Block A를 건너뛰면?

Q. Block A(고전 ML)를 안 배우고 Block B(신경망)로 바로 넘어가면 안 되나요?

- 이론적으로는 되지만, **권장하지 않는다**.
- Week 6 정규화, Week 7 트리 앙상블의 **"편향-분산 트레이드오프"**와 "train/val/test 분리"는 신경망에도 **그대로 적용** — Week 9의 드롭아웃·배치정규화는 정확히 같은 원리.
- 지금도 정형 데이터에서는 고전 ML이 신경망을 이기는 경우가 흔해서 **실무에서 바로 쓸모**가 있다.

Q. 수학이 약한데 따라갈 수 있나요?

- 1.3절의 세 가지 수학(내적, 연쇄법칙, 베이즈 정리)이 **사실상 전부**. 그 이상의 새 수학 도구는 거의 등장하지 않고,
- 대신 그 세 가지를 **점점 더 복잡한 상황에 반복 적용**하는 것이 이번 학기의 실제 난이도. 낯선 표기가 나올 때마다 1.3절을 참고하는 습관이면 충분.

자주 묻는 질문 3: 순서와 깊이

Q. ML2를 먼저 듣고 ML1을 나중에 들어도 되나요?

- **된다** — 상호 선행 과목 관계가 아니라 완전 독립 설계. ML2는 1주차에 신경망 기초를 압축해서 다시 짚어줌.
- 다만 ML1의 "정형 데이터에서 왜 아직도 고전 ML이 강한가" 실무 감각은 ML2(로봇·강화학습 중심)에서는 다루지 않음 — 산업 현장에서 둘 다 쓸 예정이라면 **순서와 무관하게 두 과목 다** 권함.

Q. 모든 장을 같은 깊이로 배워야 하나요?

- 아니다 — 목표는 16개 장을 동일하게 깊게 이해하는 것이 아니라, 매 장에서 **"h는 무엇이고, J는 무엇이며, 어떻게 최소화하는가"**라는 세 가지 질문에 답할 수 있는 정도.
- 특정 장(예: Week 12)이 흥미로우면 더 깊게 파도 좋지만, 그때도 Week 9의 역전파가 먼저 잡혀 있어야 "깊이 파는 것"과 "그저 보는 것"을 스스로 구분할 수 있다.

Python이 약한데? + 이번 차시 마무리

Q. Python이 약한데 코드를 따라갈 수 있나요?

- 기본 문법(변수, for 루프, 함수, 리스트)만 있으면 충분.
- 이 책은 의도적으로 개념 코드를 **순수 Python**(리스트 + for 루프)으로 짠 경우가 많음 — "벡터 연산 뒤에 숨은 for 루프를 직접 눈으로 보기"가 우선.
- scikit-learn · PyTorch의 호출 이름을 외우는 것이 목표가 아니라, "**이 함수가 안에서 뭘 하는가**"를 알고 쓰는 것이 목표.

1.1 마무리

어떤 모델을 쓸지보다 먼저 물어야 할 질문은 항상 같다: **이 문제는 무엇을 데이터로 갖고 있고, 무엇을 맞히고 싶은가.**

1.2 머신러닝 문제 정식화와 세 갈래 분류

문제를 푸는 게 아니라, 먼저 "정의"부터

"어떤 알고리즘을 쓸까"부터 고민하는 것이 흔한 착각 — 실전에서는 **순서가 반대**. 알고리즘보다 먼저 문제 자체를 세 가지로 정의한다:

1. **입력** x — 무엇을 **특징(feature)**으로? (예: 집의 평수, 방 개수, 지어진 연도)
2. **출력** y — 무엇을 맞히나? **연속값(회귀)**인가 **범주(분류)**인가?
3. **가설** h — 입력$\rightarrow$출력 함수를 어떤 **형태**로? (직선? 트리? 신경망?)

집값 예측: x=평수·방·위치, y=거래가 (**연속값**), h=**선형 함수**부터. 세 가지가 안 정해지면 학습 목표 자체가 불분명.

스팸 필터: x=제목의 '무료'·발신자·느낌표 수 (**특징의 벡터**), y=스팸(1)/정상(0)(**범주**). x, y 의 **형태(연속 vs 범주)**가 다르니 **손실함수도 달라야** 한다.

x 를 잘못 고르면(예: 스팸 필터에 "도착 요일"만 특징으로) 아무리 좋은 알고리즘을 써도 h 가 배울 **신호 자체가 없다**. **"어떤 알고리즘이 가장 정확한가"보다 "이 특징들이 애초에 정답과 관련이 있는가"가 항상 먼저 물어야 할 질문.**

학습(training)의 표기: m 과 (i)

학습의 정의

주어진 데이터 $(\boldsymbol{x}^{(1)}, y^{(1)}), \ldots, (\boldsymbol{x}^{(m)}, y^{(m)})$ 에 대해 $h(\boldsymbol{x}^{(i)})$ 가 $y^{(i)}$ 에 최대한 가깝도록 h 의 **파라미터를 조정**하는 과정.

- 위 첨자 (i) = "i 번째 데이터 샘플" (이 학기 내내 반복되는 표기). $\boldsymbol{x}^{(3)}$ = 세 번째 집의 특징 벡터.
- m = 학습 데이터의 **전체 개수**(샘플 수).
- 예: 집 200채면 $m = 200$, $\boldsymbol{x}^{(37)}$ = 37번째 집의 (평수, 방 개수, 연식), $y^{(37)}$ = 그 집의 실제 거래가.

학습은 "학습 데이터를 **외우는 것**"이 아니라, **본 적 없는 201번째 집에도 일반화되는 규칙을 찾는 것** — 일반화 능력의 엄밀한 검증은 Week 6.

실제 예: Kaggle Housing Prices — m과 (i)

- **Housing Prices Dataset** (Kaggle) — 집 $m = 545$채, 각 집마다 특징 12개 + 정답 가격 1개.
- **특징** x: 면적(area), 침실 수(bedrooms), 욕실 수 (bathrooms), 층수(stories), 주차공간(parking), 대로변 여부 (mainroad) 등.
- **타깃** y: 집값(price).

표기	의미
$m = 545$	데이터셋에 있는 **집의 개수**
$\boldsymbol{x}^{(1)}$	첫 번째 집의 특징들 (면적, 침실 수, ...)
$y^{(1)}$	첫 번째 집의 **실제 가격**
$\boldsymbol{x}^{(i)}, y^{(i)}$	i 번째 집의 특징과 가격 ($1 \le i \le m$)

출처: Housing Prices Dataset, Kaggle (kaggle.com/datasets/yasserh/housing-prices-dataset).

“정의를 먼저”의 역사: 세 시대

“먼저 문제를 정의하라”가 추상적 교훈이 아니라 실제 역사에서 어떤 결과를 만들었는지 세 시대로:

- **1959 사무엘의 체커스 (Samuel, 1959)**: “머신러닝”이라는 말을 현대적 의미로 만든 논문. 기계를 “플레이하게” 하는 게 아니라, “**이 판국이 얼마나 좋은가**(평가함수, y)를 **학습**하는 문제”로 **재정의** — 판국 수가 사람이 규칙을 전부 쓸 수 없을 만큼 많으므로 기계가 self-play에서 배우게. (이 재정의가 40년 뒤 **AlphaGo 계보**의 출발점.)
- **1970s 전문가 시스템 (MYCIN, 1976) — 역의 길**: 의사들이 “X 증상이면 Y를 의심하라”는 **규칙을 수백 개 직접** 써넣은 감염 진단. 좁은 영역에서 전문의에 필적했으나, 규칙 **유지보수가 사람에게**, 새 영역마다 수천 개를 다시 사람이 — 교훈: “정의(=모든 규칙을 사람이 쓰는 것)가 사람에 의존할수록, 상한도 사람의 역량에 갇히고 유지비도 비싸진다.”
- **1998 LeNet과 우편 자동화 (Week 10)**: “입력: 손글씨 숫자 이미지, 출력: 어느 숫자인가”. 사람이 형상(입력/출력)과 특징을 설계하고, “특징을 **어떻게 쓸 것인가**”는 **기계가 데이터에서 학습** — USPS 우편 분류에서 “사람은 문제만 정의하고 **기계가 세부 규칙을 배운다**” 경로가 **확장성에 이긴 첫 대규모 증거**.

세 시대가 주는 메시지

"정의를 먼저 하라"는 매핑($x \to y$)의 어느 부분을 사람이 쓰고, 어느 부분을 기계가 배우게 할 것인가를 정한다는 뜻.

- "평가함수를 쓴다" $\to$ "학습한다" (1959, 사무엘)
- "인식 규칙을 쓴다" $\to$ "특징을 학습한다" (1998, LeNet)
- "정의를 바꾸는" 순간 불가능했던 문제가 가능해졌다.

실습: 세 문제를 스스로 정식화해보기

각 시나리오에 대해 x(입력 특징), y(출력), h 의 대략적 형태(직선? 규칙 기반? “이번 학기에 배울 것”)를 노트에 적어보기.

시나리오	힌트 (x / y / 갈래)
1. 신용카드 이상거래 탐지 매 결제가 사기인지를 즉시 판단	x=결제 금액·시간·가맹점 종류, y=사기/정상(범주) $\rightarrow$ **???**
2. 뉴스 기사 주제 자동 분류 정치/경제/스포츠/문화 태그	x=기사 텍스트(나중에 단어 빈도·임베딩으로), y=4개 범주 중 하나 $\rightarrow$ **???**
3. 공장 설비 잔여수명 예측 진동·온도·가동시간 센서	x=센서 측정값들, y=남은 일수(숫자) $\rightarrow$ **???**

세 문제 모두 “**정답 라벨이 있고 y 가 범주냐 숫자냐**”라는 같은 질문 하나로 갈린다 — 이번 학기 내내 새 문제를 만날 때마다 가장 먼저 꺼내 쓸 도구.

실습: 세 문제를 스스로 정식화해보기 (정답)

각 시나리오에 대해 x(입력 특징), y(출력), h 의 대략적 형태(직선? 규칙 기반? “이번 학기에 배울 것”)를 노트에 적어보기.

시나리오	힌트 (x / y / 갈래)
1. 신용카드 이상거래 탐지 매 결제가 사기인지를 즉시 판단	x=결제 금액·시간·가맹점 종류, y=사기/정상(범주) $\rightarrow$ **분류**
2. 뉴스 기사 주제 자동 분류 정치/경제/스포츠/문화 태그	x=기사 텍스트(나중에 단어 빈도·임베딩으로), y=4개 범주 중 하나 $\rightarrow$ **분류(다중 클래스)**
3. 공장 설비 잔여수명 예측 진동·온도·가동시간 센서	x=센서 측정값들, y=남은 일수(숫자) $\rightarrow$ **회귀**

세 문제 모두 “**정답 라벨이 있고 y 가 범주냐 숫자냐**”라는 같은 질문 하나로 갈린다 — 이번 학기 내내 새 문제를 만날 때마다 가장 먼저 꺼내 쓸 도구.

“정답”은 언제나 사람이 정한다: 라벨의 책임 I

$y^{(37)}$ 이 “그 집의 실제 거래가”라고 썼는데, “실제”를 오래 바라보자 —

- $y^{(37)}$ 은 “세상에서 정말 그 집의 가치”가 아니라, **그 거래가 기록된 숫자**. 특수 사유(가족 간 거래, 급매)로 비정상적으로 싸게 거래됐다면 “가치”는 달라진다.
- 즉 라벨은 “세상의 진실”이 아니라 **“그 시점, 그 상황에서 사람이 기록한 값”**.

실무 결과 1: 라벨의 정의는 팀 전체가 합의해야 한다

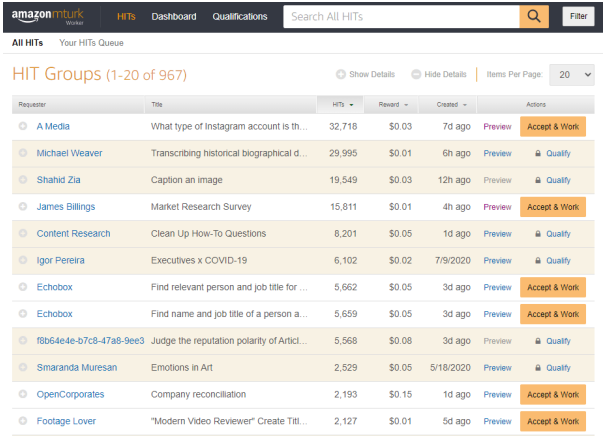
“이탈 = 30일 이상 미접속”으로 정한 팀과 “이탈 = 구독 해지”로 정한 팀이 같은 데이터셋에 라벨을 매기면, **같은 고객에게 두 라벨이 동시에** 붙을 수 있다. 정의가 일관성이 없으면 아무리 좋은 h 도 그 모순을 학습할 수 없다.

실무 결과 2: 라벨러 한 명은 못 믿는다 — 다수결로 합의

- 라벨러 한 명의 판단은 **주관적**일 수 있다 — 같은 X-ray를 의사 두 명이 다르게 읽는 일도 흔하다.
- 해결: 같은 데이터를 **3명 이상**이 **독립적으로** 라벨링 $\rightarrow$ **다수결(majority vote)** 또는 합의로 최종 라벨 확정.
- 라벨러들이 서로 **얼마나 동의하는지**(inter-annotator agreement)를 Cohen's/Fleiss' κ 로 수치화 — κ 가 낮으면 데이터가 아니라 **라벨 정의 자체가 모호**하다는 신호.
- 예: ImageNet은 Amazon Mechanical Turk 라벨러 다수의 합의로 각 이미지의 클래스를 확정했다.

출처: Snow et al., "Cheap and Fast—But is it Good? Evaluating Non-Expert Annotations for Natural Language Tasks," EMNLP 2008; Deng et al., "ImageNet: A Large-Scale Hierarchical Image Database," CVPR 2009.

실제 예: Amazon Mechanical Turk의 라벨링 작업 큐



요청자(Requester)가 “Caption an image”, “Judge the reputation polarity” 같은 작은 작업(**HIT**, Human Intelligence Task)을 몇 센트에 올리면, 여러 명의 워커(Worker)가 **같은 작업을 독립적으로** 수행 — 이 결과들을 다수결/합의로 합쳐 최종 라벨을 만드는 것이 앞 슬라이드의 “3명 이상 합의”가 실제로 돌아가는 모습이다.

라벨의 책임 II — 품질 확인과 세 갈래의 기준

실무 결과 3: 라벨 품질은 EDA 단계에서 먼저 확인

라벨 분포가 **99:1**로 극단적으로 불균형하면 "정확도 99%"는 사실상 **무의미**하다(전체를 "비정상"으로 예측해도 99%). 해결은 라벨 재정의 · 재가중치 · 다른 평가지표(PR-AUC, Week 2.3) — 출발점은 모두 "**이 라벨이 정확히 무엇을 측정하는가**"를 처음에 명확히 했는가.

- 이 개념이 바로 다음 세 갈래 분류와 직결된다: **라벨이 있는가 없는가가 지도/비지도/강화를 가르는 기준**이기 때문이다.

세 갈래 분류: 무엇을 데이터로 갖고 있는가

머신러닝 문제는 세 갈래로 나뉜다 — 알고리즘의 이름이 아니라, **어떤 형태의 데이터를 갖고 있고 어떤 신호로 학습하는가**로 갈린다.

	데이터 형태	목표	이번 학기 예시
지도학습	(x, y) 쌍	새 x에 대한 y 예측	회귀·분류, kNN, SVM, 트리, 신경망
비지도학습	x 만	데이터의 숨은 구조 발견	k-means, PCA, EM/GMM
강화학습	상태·행동·보상	누적 보상을 최대화하는 정책	**ML1에서는 안 다룸** (ML2 전담)

강조: 기준은 "얼마나 어려운 문제인가"가 아니라 **"손에 쥔 데이터에 무엇이 적혀 있는가"**.

지도학습 내부: 판별적 vs 생성적

지도학습 안에서도 다시 두 갈래가 갈린다:

판별적 (discriminative)

- $P(y \mid x)$ — "x 가 주어졌을 때 y 의 확률"을 **곧바로** 학습.
- 예: 회귀 · 로지스틱회귀 · SVM · 신경망.

생성적 (generative)

- $P(x \mid y)$ — "각 클래스가 데이터를 **어떻게 만들어내는지**"를 먼저 학습.
- 그 다음 **베이즈 정리로 뒤집어** $P(y \mid x)$ 를 얻는다. 예: Week 3의 나이브베이즈 · GDA.

같은 문제를 **정반대 방향**에서 접근하는 두 철학의 차이는 Week 3에서 자세히 다룬다.

여기서 "생성적"은 **분류**를 위해 $P(x \mid y)$를 학습하는 것 — 이미지 · 텍스트를 새로 만들어내는 Week 15의 **생성형 모델**(VAE·GAN·Diffusion, LLM 포함)과는 별개의 과제이며, 지도학습 분류에 속하지 않는다. 다만 "데이터가 어떻게 만들어지는지"를 모델링한다는 철학은 같아서, 이 관점이 그 생성형 모델들로 이어지는 첫 걸음이다.

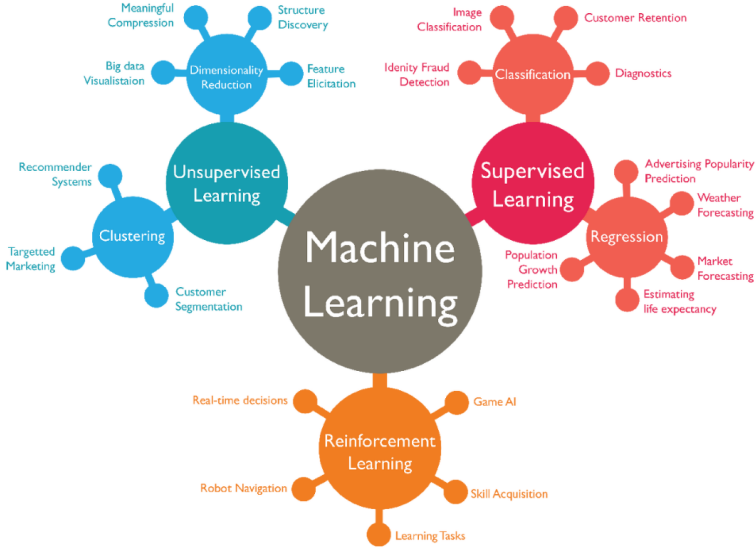
비지도학습과 강화학습

비지도학습 — 정답 없이 **구조만** 주어짐

- “이 고객들을 비슷한 성향끼리 묶어라”(**군집화**).
- “1000개 특징을 정보 손실 최소로 10개로 줄여라” (**차원축소**).
- Week 14∼15에서 다룸.

강화학습 — 정답이 아니라 **보상만** 주어짐

- 에이전트가 **시행착오**로 좋은 행동을 스스로 찾음.
- 체스·바둑: “이 수가 정답이다”라고 알려주는 사람은 없지만, “이겼다/졌다”는 신호는 있다.


Machine Learning
Unsupervised Learning
Dimensionality Reduction
Meaningful Compression
Structure Discovery
Big data Visualistaion
Feature Elicitation
Clustering
Recommender Systems
Targetted Marketing
Customer Segmentation
Supervised Learning
Classification
Image Classification
Customer Retention
Idenity Fraud Detection
Diagnostics
Regression
Advertising Popularity Prediction
Weather Forecasting
Population Growth Prediction
Market Forecasting
Estimating life expectancy
Reinforcement Learning
Real-time decisions
Game AI
Robot Navigation
Skill Acquisition
Learning Tasks

두 질문을 실제 시나리오에

시나리오	Q1: y 가?	Q2: 보상이?	갈래
지난 분기 매출로 다음 분기 매출 예측	있음(매출 숫자)	없음	**???**
고객 구매 기록만으로 "비슷한 취향" 그룹 발견	없음	없음	**???**
드론이 충돌 없이 목표지에 도착하도록 경로 학습	없음	있음(도착/ 충돌 신호)	**???**
X-ray 사진으로 폐렴 유무 판별	있음(의사 진단)	없음	**???**

왜 두 단계인가

y 가 있으면 강화학습의 구조(상태·행동·보상)가 존재할 여지가 **아예 없다** $\Rightarrow$ 질문 1이 먼저. 2를 먼저 던지면 y 가 있는 데이터에서 보상을 **억지로** 만들 위험 — "매출이 올랐으니 보상 +1"의 순간, **보상 설계(reward shaping)**의 가장 흔한 함정(행동$\rightarrow$결과 인과가 모호).

두 질문을 실제 시나리오에 (정답)

시나리오	Q1: y 가?	Q2: 보상이?	갈래
지난 분기 매출로 다음 분기 매출 예측	있음(매출 숫자)	없음	지도·**회귀**
고객 구매 기록만으로 "비슷한 취향" 그룹 발견	없음	없음	**비지도**
드론이 충돌 없이 목표지에 도착하도록 경로 학습	없음	있음(도착/ 충돌 신호)	**강화**
X-ray 사진으로 폐렴 유무 판별	있음(의사 진단)	없음	지도·**분류**

왜 두 단계인가

y 가 있으면 강화학습의 구조(상태·행동·보상)가 존재할 여지가 **아예 없다** $\Rightarrow$ 질문 1이 먼저. 2를 먼저 던지면 y 가 있는 데이터에서 보상을 **억지로** 만들 위험 — "매출이 올랐으니 보상 +1"의 순간, **보상 설계(reward shaping)**의 가장 흔한 함정(행동$\rightarrow$결과 인과가 모호).

세 갈래를 대표하는 유명한 시스템

- **지도 — ImageNet/AlexNet**: 100만 장이 넘는 사진 각각에 사람이 미리 "고양이"·"자동차" 같은 **정답 라벨**을 붙여뒀다. 라벨이 없었으면 이 방식 자체가 **불가능**.
- **비지도 — 넷플릭스/스포티파이 군집화**: "당신과 비슷한 취향의 사람이 좋아한 콘텐츠"를 추천할 때, "이 사람은 그룹 3에 속한다"는 **정답 라벨은 어디에도 없다**. 시청·청취 기록만으로 비슷한 패턴을 알고리즘이 스스로 묶어냄.
- **강화 — AlphaGo**: 바둑의 "정답 수"를 사람이 알려준 적 없다(최선의 수를 사람도 모른다). 대신 "**이겼다(+1)/ 졌다(-1)**"라는 보상만으로 스스로 수백만 판을 두며 정책 개선 — ML2에서 처음부터 다룸.

같은 데이터, 다른 갈래: "라벨 관점"이 문제를 바꾼다

한 이커머스: 각 고객의 (연령, 지역, 월 구매 횟수, 평균 구매액, 최근 90일 방문 빈도) + "지난 30일 구독 해지 여부(1/0)" 라벨.

1. **그냥 있는 그대로** — y = 해지 여부 예측 → **지도학습 · 분류**(이탈 예측 모델, Week 2∼7의 도구 중 하나).
2. **라벨만 버리면** — 특징만으로 "비슷한 구매 성향을 묶어라" → **비지도학습 · 군집화**. 군집이 나중에 "이 군집의 이탈 확률이 높다"는 통찰로 이어질 수도 — **라벨을 버린 비지도가 라벨을 쓴 지도보다 넓은 시야**를 줄 수 있음.
3. **한 걸음 더** — 매 주 쿠폰(10%/20%/무료배송)을 직접 결정하고, 다음 주 매출(보상)을 관찰하며 **시행착오**로 개선 → **강화학습**. "이 고객에게 이 쿠폰이 정답"이라는 라벨은 어디에도 없고, **보상(매출 변화)만**.

같은 원본 데이터셋에서, 무엇을 라벨로 볼 것인가에 따라 세 갈래 모두가 만들어진다.

ML1의 주차가 지도(Week 2∼13) + 비지도 (Week 14∼15) 두 갈래에 머무는 이유: 강화학습은 "정답이 아예 없고 보상만 있다" — 데이터의 형태 자체가 **근본적으로 달라** 별도 과목(ML2)을 통째로 할애할 만큼 다르다.

회귀 vs 분류: 같은 모델, 다른 출력

지도학습은 다시 출력이 **연속값이냐 범주냐**로 나뉜다:

- **회귀(regression)**: **숫자**를 예측 (집값, 기온).
- **분류(classification)**: **범주**를 예측 (스팸/정상, 개/고양이/새).

같은 선형 모델 $w^\top x + b$ 라도, **출력을 그대로 쓰면 회귀**, **시그모이드를 씌워 0∼1 확률로 해석하면 분류** (둘 다 Week 2). 앞으로 몇 장 동안 반복되는 패턴:

모델 구조는 비슷한데, 출력을 해석하는 방식과 손실함수(loss function)만 문제에 맞게 바뀐다.

- 예: 학습된 $w = [0.5, -2.0]$, $b = 1.0$, 새 데이터 $x = [3, 0.5]$.

$$w^\top x + b = 0.5 \times 3 + (-2.0) \times 0.5 + 1.0 = 1.5 - 1.0 + 1.0 = 1.5$$

같은 1.5, 두 가지 읽기 — 시그모이드

- **회귀로** 쓰면: 결과 1.5 를 **그대로 예측값**으로 읽음 (예: "이 집은 1.5억 원").
- **분류로** 쓰면: 정확히 같은 1.5 를 시그모이드 $\sigma(z) = 1/(1+e^{-z})$ 에 통과시켜 $\sigma(1.5) \approx 0.82$ $\rightarrow$ "이 이메일이 **스팸일 확률 82%**"로 읽음.

가중치 w, 편향 b, 입력 x 는 완전히 동일한데, 오직 "출력을 어떻게 해석할 것인가"만 바뀌었다.

시그모이드가 왜 필요한가

z 는 $(-\infty, +\infty)$ 의 아무 값이나 나온다. "확률"로 읽으려면 0과 1 사이가 필요하고, z 가 커질수록 1에, 작을수록 0에 **부드럽게(미분 가능하게)** 가까워져야 한다 — 그래야 Week 2 경사하강법에서 미분이 잘 되기 때문. 시그모이드는 이 두 성질을 동시에 만족하는 **가장 간단한** 함수.

링크 함수 + 실습: 같은 계산, 두 가지 해석

"출력 해석을 확률로 바꾸는" 단계 = **링크 함수**: 회귀 = **항등 함수**(z 를 그대로), 이진 분류 = **시그모이드**. Week 2에서 이 차이로 **손실함수가 어떻게 달라지는지** 직접 유도(MSE vs 교차 엔트로피).

```
import math

def sigmoid(z):
    return 1 / (1 + math.exp(-z))

w = [0.5, -2.0]
b = 1.0
x = [3, 0.5]

z = sum(wi * xi for wi, xi in zip(w, x)) + b
print("z =", z)                          # 1.5
print("회귀로 읽으면:", z)                # 1.5 그대로( 예측값)
print("분류로 읽으면:", sigmoid(z))       # 약 0.818 양성( 클래스일확률 )
```

z 를 만드는 코드는 **한 줄로 똑같고**, `sigmoid()` 를 씌우느냐 마느냐 **한 줄 차이**가 회귀/분류. Week 2: 이 z 를 만드는 w, b 자체를 **데이터로 학습**(경사하강법). Colab: 2D 점군을 회귀선 vs 분류 경계로 나란히, `classify_dataset()` 작성.

공통 뼈대: 손실함수 J 를 정의하고 최소화한다

지금부터 배우는 거의 모든 알고리즘은 다음 **두 단계 틀**을 따른다:

1. 모델이 얼마나 틀렸는지를 **숫자 하나로 요약**하는 **손실함수** $J(w)$ 를 정의.
2. $J(w)$ 를 **최소화**하는 w 를 찾는다 (대개 **경사하강법**, Week 2).

- 회귀의 손실(평균제곱오차)과 분류의 손실(교차 엔트로피)은 **형태가 다르지만**, "틀린 정도를 수치화하고 그걸 줄인다"는 **구조 자체는 동일**.
- Week 2 선형·로지스틱회귀, Week 5 SVM(힌지 손실), Week 9 신경망 (역전파로 손실 최소화)까지 — h 의 형태와 J 의 모양은 알고리즘마다 다르고, **절차 자체는 똑같이 반복**.

매 주차에 습관적으로 던질 세 질문: "h 는 무엇이고, J 는 무엇이며, 어떻게 최소화하는가."

세 갈래를 가르는 “데이터의 형상” — 표로 보면 된다

- **지도**: **2개 블록** — 왼쪽 x 블록(특징 열), 오른쪽 y 블록(라벨 1열). 행 m 개, 열 $(p+1)$ 개. 핵심 질문: “라벨 열이 있는가?”
- **비지도**: **1개 블록** — x 블록만 있고 라벨 **열이 없다**. 행 m 개, 열 p 개. 목표: “구조를 찾자”.
- **강화**: 형상 자체가 다름 — 각 행이 (s_t, a_t, r_t, s_{t+1}) 4개 필드(상태, 행동, **보상**, 다음 상태)를 가진 **경험 튜플** 목록. 라벨 열 대신 **보상 열**이 있다.

“라벨 열”과 “보상 열”의 차이

라벨 = “이 입력 x 에 대한 **정답**”. 보상 = “이 행동을 취한 **결과**가 좋았는가”를 숫자로 알려줄 뿐, “정답 행동은 무엇이었는가”를 **아주 느리고 부분적으로**만 알려준다. 이것이 세 갈래의 가장 근본적인 차이 (ML2에서 본격 다룸).

자주 하는 실수: 정의를 건너뛰고 알고리즘부터

- 사례: 스타트업이 "이탈 예측 모델 만들자"는 목표만 갖고 "**그럼 신경망을 쓸까, XGBoost를 쓸까?**"부터 논의.
- 몇 주 뒤 모델은 완성됐지만 **성능이 엉망** — "이탈"의 정의(30일 미접속? 구독 해지? 환불 요청?)조차 **팀마다 다르게** 이해하고, y(라벨) 자체가 일관성 없이 매겨져 있었음.

알고리즘 선택에 쓴 시간보다, y 를 정의하는 데 쓴 시간이 훨씬 적었던 것이 진짜 원인.

- "어떤 모델이 가장 정확할까"는 **프로젝트 중반 이후**의 질문. 초반에 가장 먼저 답해야 할 질문은 "x 는 무엇이고, y 는 정확히 무엇이며, 그 정의에 **모두가 동의하는가**".
- Week 8, 16 팀 프로젝트: 데이터를 내려받자마자 모델링에 뛰어들지 말고, 이 세 가지 질문에 **팀 전체가 명시적으로** 답을 적어두기.

실수 두 번째: “라벨 없으니 비지도겠지”

“라벨이 없다”는 **현상**이 “비지도학습이 맞는 문제”라는 **결론**으로 바로 이어지지는 않는다.

- **“라벨이 비싸서 못 만들었다” vs “라벨이 원리적으로 없다”**: X-ray 폐렴 판별에서 “폐렴/정상” 라벨은 의사가 매겨야 해 비싸고 느리다. 비지도로 푸는 것은 “라벨을 만들지 않겠다”는 선택이지 “원리적으로 없다”가 아님 $\rightarrow$ 대개는 “**라벨을 일부라도** 만들 수 있는가”를 먼저 고민 (의사 10장만 라벨 + 나머지 비지도 = **반지도학습**, Week 9∼10에서 간접 다룸).
- **“보상이 있긴 한데 ’라벨’인 줄 알았다”**: “재구매 여부”는 **라벨**(지도)로 쓸 수도, “또 샀으니 +1”이라는 **보상**(강화)으로 읽을 수도 — Q1/Q2로 다시 확인.

확인 문제: 이 시나리오는 어떤 갈래인가 (a)∼(d)

각 사례를 읽는 지도/비지도/강화, 그리고 회귀/분류까지 판단:

시나리오	답
(a) 넷플릭스가 시청 기록(장르, 시청 시간)만으로 "비슷한 취향 그룹" 자동 탐색	**비지도** — "그룹 3에 속한다"는 정답 없이 구조(비슷한 패턴)만으로 탐색
(b) 은행이 과거 대출자(소득, 신용점수, 상환 여부)로 "새 신청자가 상환할지" 예측	**지도·분류** — "상환함/못함" 범주 정답이 이미 붙어 있음
(c) 로봇 청소기가 배터리 아끼며 방을 최대한 깨끗이 청소하도록 스스로 수백 번 시행착오	**강화** — "이 방향이 정답"을 알려주는 사람 없이 "청소 면적"·"남은 배터리" 보상으로 정책 개선 (ML2에서 본격)
(d) 부동산 플랫폼이 매물 정보로 예상 거래가를 **숫자**로 예측	**지도·회귀** — y(거래가)가 범주가 아니라 연속 숫자

기준이 항상 같다: "정답 라벨이 있는가", 있다면 "숫자인가 범주인가" — 질문 두 개로 어떤 문제든 분류할 수 있다.

확인 문제 (e)(f)(g) + 1.2 마무리

- **(e)** 게임사가 "플레이어가 다음 주에도 접속할 것인가"를 과거 행동 로그로 예측 → **지도·분류** ("다음 주에도 접속" **라벨**이 이미 붙어 있음).
- **(f)** 같은 게임사가 "지금 어떤 아이템을 쓰면 장기 승률에 좋은지"를 실제 플레이(시행착오)로 학습 → **강화** ("지금의 정답 행동" 라벨은 없고 "승리/패배" **보상**만). (e)(f)는 **같은 게임 로그**를 갖고, 예측 대상을 보는 관점에 따라 갈래가 바뀜.
- **(g)** SNS가 친구 관계 **그래프만** 갖고 "비슷한 커뮤니티"를 자동 탐색 → **비지도** ("커뮤니티 5에 속한다"는 라벨 없이 연결 밀도 구조로 탐색).

Q. "지도 vs 비지도"와 "회귀 vs 분류"는 같은 기준인가? 아니다 — **수직** 관계. 지도가 "라벨이 있는가?"로 먼저 갈리고, 그 안에서 "숫자인가 범주인가?"로 갈린다. 비지도/강화는 회귀/분류 구분 **그 자체가 없다**.

반지도(라벨 1% 수준)·자기지도(라벨이 데이터에서 자동 생성)는 세 갈래의 **경계** 변종 (Week 9∼10, 14∼15에서 간접 등장). "보상"이 있어도 무조건 강화가 아님 — **보상의 인과성**(행동→결과)이 성립할 때만 신호 (ML2 reward shaping).

1.2 마무리

핵심 질문

어떤 문제를 만나든, 첫 질문은 항상 같다: **손에 쥔 데이터에 무엇이 적혀 있는가 — 그리고 그것을 "라벨"으로 볼 것인가, "보상"으로 볼 것인가, 아니면 "구조"로만 볼 것인가.**

- x, y, h 로 **먼저 정의**하고, 두 질문으로 세 갈래를 가르고, 지도 안에서는 "연속/범주"로 회귀/분류를 가른다.
- 회귀 vs 분류 = **같은 모델, 다른 출력** (항등 링크 vs 시그모이드).
- 모든 알고리즘의 뼈대 = J **를 정의하고 최소화** (h, J, 최소화 방법의 세 질문).

1.3 데이터사이언스 파이프라인과 사전지식 리뷰

모델은 파이프라인의 한 조각일 뿐

1.2절은 "모델을 어떻게 정의하는가"였지만, 실전에서 모델 학습은 전체 작업의 **작은 일부**에 불과하다. 실제 데이터 프로젝트의 순환:

1. **수집** — 어디서 데이터를 구할 것인가. Week 8, 16 팀 프로젝트에서는 Kaggle, UCI ML Repository, Hugging Face Datasets 등 공개 저장소를 주로 사용. 실무에선 "이 데이터를 이 목적으로 써도 되는가"(라이선스, 개인정보) 확인도 포함 — 캐글 대회 데이터라도 **상업적 재사용이 금지**된 경우가 흔함.
2. **정제** — **결측치** · **중복** · **이상치**를 처리 (현실 데이터는 거의 항상 지저분).
3. **탐색적 데이터 분석(EDA)** — 모델을 만들기 전에 분포·상관관계·클래스 비율을 **눈으로** 확인. 이 단계의 발견이 어떤 모델을 고를지까지 결정 (예: 99:1 불균형 $\rightarrow$ 정확도 대신 PR-AUC, Week 2.3).
4. **모델링** — 1.2절에서 정식화한 문제에 맞는 h 를 고르고 학습. 이번 학기 대부분의 장.
5. **평가** — **학습에 쓰지 않은** 데이터로 성능 검증 (엄밀한 원리는 Week 6). 나쁘면 4단계(다른 모델?)로만 돌아가지 않고 **1∼3단계로 되돌아가야** 할 수도 — "모델이 나쁜 것"이 아니라 "데이터가 애초에 그 신호를 담고 있지 않은 것"일 수 있기 때문.

왜 이 순서인가 — 결함은 아래로 "증폭"된다

- **모델링은 다섯 단계 중 하나일 뿐** — 처음 배우는 이들이 가장 자주 간과하는 점. 캐글 프로젝트에서 결측치 처리 + EDA에 드는 시간이 모델 코드보다 **긴 경우가 흔함**.
- 파이프라인 앞 단계의 결함은 뒤에서 아무리 좋은 모델을 써도 **수리되지 않는다**.

- **결측치 미처리**: 모델이 "결측 처리의 흔적"을 **정답인 양** 학습.
- **중복 행**: 같은 사례를 여러 번 외워 "중요도가 부풀려진" 데이터.
- **미래 정보**(예: "거래 완료 여부"를 특징으로 쓰는 집값 예측): 훈련 때는 비현실적으로 정확하고 **실전에만 무너지는**, 디버깅이 거의 불가능한 모델.

실무 경험자들의 일치된 증언

파이프라인은 위에서 아래로 흐르되, **결함은 위에서 아래로 "증폭"**되는 구조.

자주 하는 실수: EDA를 건너뛰고 바로 모델링

1.2절의 “정의 없이 알고리즘부터 고르지 말라”가 이 파이프라인에도 그대로 적용된다.

- 흔한 실패 패턴: 데이터를 내려받자마자 `df.head()` 만 한 번 찍어보고 바로 `model.fit(X, y)` 를 돌림.
- 나중에 성능이 이상해서 다시 들여다보면:
  - 가격 열에 **음수값**이 섞여 있었거나,
  - 한 지역이 전체의 **90%**를 차지해 다른 지역은 사실상 학습이 안 됐거나,
  - 특정 열이 **“미래 정보”**(예측 시점엔 알 수 없는 값) 를 담고 있어 학습 때만 비현실적으로 정확했던 것.

EDA 10분의 가치

전부 `df.describe()`, `df.corr()`, 그룹별 통계 몇 줄 만 먼저 찍어봤어도 모델을 돌리기 **전에** 잡을 수 있었던 것. EDA에 쓰는 10분이, 잘못된 가정 위에 쌓은 모델을 디버깅하는 **몇 시간**을 아껴준다.

유명한 사고: Aha! 데이터셋 (Wolpert & Kibler, 1992)

머신러닝 역사에서 **데이터 품질**의 중요성을 보여주는 가장 유명한 사례:

- 1992년, SRI 국제 AI 센터의 **Wolpert와 Kibler** — “Do We Know the Data We’re Using?” (Wolpert & Kibler, 1992).
- 당시 표준 벤치마크 UCI의 “Aha!” 데이터셋을 들여다봤더니, **동일한 특징 벡터의 행이 서로 다른 정답 라벨로 2개 이상** 들어 있는 경우가 **100건**.
- 같은 입력에 서로 모순되는 정답이 두 개면, 아무리 완전한 모델이라도 그 쌍 중 **적어도 한 건은 틀릴 수밖에** 없다 (수학적 문제).

교훈

“어떤 알고리즘이 더 좋은가” 비교 논문들에, 알고리즘이 아니라 **데이터의 모순이 만든 “반드시 틀려야 하는” 바닥**이 이미 포함돼 있었다. 중복을 정리하자 정확도가 올랐다 — 성능이 이상할 때 **첫 의심은 “모델이 약해서”가 아니라 “데이터에 문제가 있어서”**일 수 있고, 그 확인은 `df.duplicated()` 같은 **단순한 검사**로 한다.

더티 데이터의 세 가지 표준 모양

"데이터가 지저분하다"는 말은 막연한 인상이 아니라, 실전에서 반복해서 나타나는 **세 가지 표준 모양**을 가리킨다 (Aha! 사례는 두 번째 모양의 극단):

1. **결측치 (missing value)**: 값이 아예 비어 있는 경우 — `NaN`, 빈칸, 혹은 0 / -1 / -999 / 999 같은 **센티넬 값**(의도된 "측정 실패" 표시). 센티넬은 "비어 보이지 않아서" **가장 위험** — 0으로 임의 채워진 결측치를 그대로 쓰면 "0은 특별한 값"이라는 **거짓 신호**가 들어감 $\rightarrow$ `describe()`의 최솟값·분포로 "합리적 범위를 벗어난 값이 모여 있는가" 확인.
2. **중복 (duplicate)**: 같은 행이 여러 번 — "완전 중복"부터 같은 응답자·동일 물건이 다른 시점에 기록된 "부분 중복"까지. `duplicated()` 는 **완전 중복만** 잡아주고, 부분 중복은 "어떤 열들의 조합을 동일 사례로 볼 것인가"를 사람이 먼저 결정 (예: 부동산 = "지번+거래일" 조합).

더티 데이터 (끝): 이상치 — 도메인 지식의 영역

- 3. **이상치 (outlier)**: 범위는 합리적이지만 분포와 **극단적으로 어긋난** 값.
- 음수 가격, 250세 나이 = 센티넬 → **1번(결측치)**으로. "합법적이지만 극단적"인 값(수백억짜리 단독주택 한 채)은 **3번(이상치)**으로 처리할지, 드물지만 **중요한 사례** 라면 그대로 둘지는 —

판단의 성격

도메인 지식으로만 판단할 수 있는 문제. "코드 짜는 것"이 아니라 "**문제를 이해하는 것**"의 영역에 속한다는 점을 기억할 것.

- 세 모양 모두 **정제 단계**에서 다루고, 발견은 **EDA 단계**에서 한다 — 다음 4가지 질문으로.

EDA: 모델을 돌리기 전에 데이터와 "대화"하기

EDA의 목적은 "데이터를 보는 것" 자체가 아니라, 모델링 **전에** 다음 질문에 답을 얻어두는 것 — 이 질문들은 **모델이 아니라 데이터**에 대해 묻는 것들이다:

1. **각 특징의 스케일이 서로 얼마나 다른가?** — 평수(수십~백) vs 연식(네 자리 연도) vs 가격(억 단위). 차이가 크면 Week 4.1의 **특징 정규화**를 모델에 넣기 전에 — EDA의 **첫** 질문.
2. **어떤 특징이 목표 변수와 강하게 연관되는가?** — 아무 특징도 관련이 없으면 예측력에 한계 (1.2절의 강조와 정확히 같은 질문).
3. **목표 변수의 분포가 합리적인가?** — 음수값, 한쪽 꼬리가 극단적으로 긴가 (가격 데이터는 보통 **오른쪽 꼬리**가 긴 분포). 꼬리가 길면 "평균을 얼마나 잘 맞히느냐" 기준(MSE, Week 2)이 극단값에 지나치게 민감해질 수 있음.
4. **클래스 균형(분류)은 적당한가?** — 양성:음성 = 99:1이면 전부를 "음성"으로 예측하는 멍한 모델도 정확도 99% — 정확도는 **무의미**하고 PR-AUC(Week 2.3)를 처음부터 염두.

실 데이터로: Kaggle "House Prices"(1,460행 × 81 특징)는 `describe()` 만 찍어도 `SalePrice` 가 **오른쪽으로 심하게 치우쳐** 있고 **최댓값이 99퍼센타일보다 수 배** 크다는 것을 즉시 보여준다 — 극단값 몇 개가 훈련을 얼마나 끌어당기는지 Week 6의 정규화를 배우기 전에 감 잡기. `describe()` · `corr()` · `isna().sum()` · `groupby()` 는 **도구**일 뿐 — EDA는 그 도구로 **질문에 답하는 과정**.

실습: pandas로 첫 EDA + 결측치와 그룹별 통계

개념 코드는 순수 Python/numpy로 손으로 짜는 경우가 많지만, 실전 데이터 탐색 단계에서는 **pandas 가 사실상 표준**.

```
import pandas as pd

data = {
    "size_sqm": [59, 84, 112, 46, 132, 78],
    "rooms": [2, 3, 4, 1, 4, 3],
    "year_built": [2015, 2008, 1998, 2020, 1995, 2011],
    "price_million": [320, 410, 505, 250, 560, 390],
}
df = pd.DataFrame(data)

print(df.describe())          # 각열의평균표준편차최소최대  /// 한눈에
print(df.corr(numeric_only=True)["price_million"])  # 목표와의연관
print(df.isna().sum())        # 열별결측개수
```

실제 데이터에는 결측치가 거의 항상 있다 — 결측 + 지역 정보로 가장 자주 쓰는 처리:

```
import numpy as np
df2 = pd.DataFrame({
    "size_sqm": [59, 84, 112, 46, 132, 78, np.nan],
    "district": ["A", "B", "A", "C", "B", "A", "C"],
    "price_million": [320, 410, 505, 250, 560, 390, np.nan],
```

dropna vs fillna: 보편 정답은 없다

`dropna()` — 결측 있는 행을 **통째로 버림**

- 결측이 **극히 일부**라면 간단하고 안전.
- 위험: 데이터가 적거나 결측이 특정 **그룹에 몰려** 있으면 그 그룹 **통째**를 지워버림 (위 예: 지역 C의 결측을 지우면 지역 C의 평균가 정보 자체가 사라짐).

`fillna()` — 중앙값·평균 등으로 **채움**

- 데이터가 적거나 결측이 특정 그룹에 몰려 있으면 **그룹 단위 통계**(지역별 중앙값)로 채우는 쪽이 안전.
- 실전 한 단계 전: "**왜** 결측인가"를 먼저 조사 — **결측 패턴 자체가 정보**인 경우 (예: "층수" 결측이 단독주택에만 몰려 있으면 "결측 여부"가 "타입"을 보여주는 특징).

데이터 나누기: 순서가 중요한 이유

EDA가 끝나면 모델링 전의 **첫 번째 실제 결정**: **train set** / **test set** 분리.

실무 원칙 (Week 6에서 엄밀하게)

데이터에 대한 **통계를 써야 하는 모든 조작**은

훈련 셋에서 fit(계산) $\rightarrow$ **테스트 셋에 apply(적용)**

순서로 — 결코 "전체에서 fit $\rightarrow$ 훈련 셋에 apply" 하지 않는다.

- 예: 가격 결측 5%를 **전체 평균**으로 채운 뒤 나눠 버리면, 훈련 행이 "**테스트 행이 포함된 평균**"으로 채워진 셈 — 실전(전체 평균을 알 수 없는 상황)에서 불가능한 정보를 훈련에 살짝 새겨넣은 것.
- 이를 **데이터 누수 (data leak)**라 부름.

데이터 누수는 작을수록 위험하다

- 5% 결측의 평균이라는 **이 정도 크기의** 누수는 **숫자에 거의 영향을 주지 않는다** — 그래서 **위험**.
- 작은 누수를 습관으로 익혀두면, 나중에 "거래 완료 여부" 같은 미래 정보를 특징에 섞어넣는 **큰 누수**와 같은 버릇으로 인식되지 않고, 어느 순간 **평가 자체가 믿을 수 없게** 되었을 때 원인 찾기가 몇 배로 어려워진다.

비율보다 중요한 원칙

나누기 비율(70:30, 80:20…)은 문제 크기에 따라 달라지지만, **테스트 셋은 마지막 평가 한 번에만 쓴다.** 테스트 성적이 나빠서 "테스트를 봐서 고치고, 다시 테스트"를 반복하면, 테스트는 "새 데이터"가 아니라 사실상 **훈련 데이터**가 되어버린다 (검증 셋의 역할은 Week 6).

실습: 파이프라인을 처음부터 끝까지 (1/2) — 정제

120채의 집, 평수 40∼140㎡, 진짜 가격은 1.5 × 평수 + 20 + 잡음(백만 원). 정제가 실제로 할 일을 확인하려 두 가지 "더티함"을 의도적으로 섞어 넣음 — **3개 완전 중복 행**, **센티넬** -999 **가 1행**.

```
import numpy as np
import pandas as pd

# 1) 수집: 지저분한데이터셋시뮬레이션
rng = np.random.default_rng(0)
size = rng.uniform(40, 140, 120)
price = 1.5 * size + 20 + rng.normal(0, 8, 120)

for i in [5, 6, 7]:                                    # 행3 완전중복
    size = np.append(size, size[i])
    price = np.append(price, price[i])
price[11] = -999.0                                     # 행1 센서센티넬
df = pd.DataFrame({"size_sqm": size, "price_million": price})

# 2) 정제행( 수준): 중복제거—통계량이관여하지않아나누기전에안전
print("rows before cleaning:", len(df))
print("min price:", df["price_million"].min())         # 가-999 보인다!
print("duplicate rows:", df.duplicated().sum())
df = df.drop_duplicates().reset_index(drop=True)
```

실습: 파이프라인 (2/2) — 나누기·채움·모델링

```
# 3) 통계량기반정제채움  () 이전에** 데이터를나눈다
perm = rng.permutation(len(df))
cut = int(0.75 * len(df))
df_tr = df.iloc[perm[:cut]].copy()
df_te = df.iloc[perm[cut:]].copy()

# 4) 채움: 훈련행에서만중앙값계산    , 그통계량을테스트에적용
med = df_tr.loc[df_tr["price_million"] >= 0, "price_million"].median()
for d in (df_tr, df_te):
    d.loc[d["price_million"] < 0, "price_million"] = med

# 5) 모델링: 선형회귀정상방정식 (, Week 에서2 유도)
def linreg(X, y):
    Xb = np.column_stack([X, np.ones_like(X)])
    w = np.linalg.solve(Xb.T @ Xb, Xb.T @ y)
    return w[0], w[1]

slope, intercept = linreg(df_tr["size_sqm"].values,
                          df_tr["price_million"].values)
```

결과: 123행 → 중복 3행 제거 → 120행, 75/25로 train 90 / test 30, train median 152.6, **learned slope=1.400, intercept=29.471** (true: 1.5, 20). 기울기 1.400 ≈ 진짜 1.5 — 0.1의 차이는 랜덤 잡음, “모델이 못 배운 것”이 아니라 **데이터 자체의 불확실성**. 절편 29.5는 진짜 20에서 더 크게 벗어남 — 표본 90개 수준에서 **절편은 기울기보다**

평가: R^2, 그리고 "잘 외운" 모델

90행 훈련 모델로 30행 테스트를 평가:

```
def r2(y, p):
    return 1 - ((y - p) ** 2).sum() / ((y - y.mean()) ** 2).sum()

def mae(y, p):
    return np.abs(y - p).mean()

Xtr, ytr = df_tr["size_sqm"].values, df_tr["price_million"].values
Xte, yte = df_te["size_sqm"].values, df_te["price_million"].values
print(f"train R2={r2(ytr, slope*Xtr+intercept):.4f}")
print(f"test  R2={r2(yte, slope*Xte+intercept):.4f}")
```

```
train R2=0.9406 MAE=7.13    test R2=0.9561 MAE=7.58
```
(평균 713만 원 오차, 평균 가격의 약 2%).

- **R^2(결정계수)**: "목표 변수의 변동 중 모델이 몇 % 설명하는가". 1.0 = 완벽, **0 = "모두 평균을 예측하는 모델"과 같은 수준**.
- 주목: **train과 test의 R^2가 거의 같다(0.94 vs 0.96)** — "일반적인 규칙"을 배운 것이지 "90행만 외운" 것이 아님을 보여주는 **좋은 신호**.

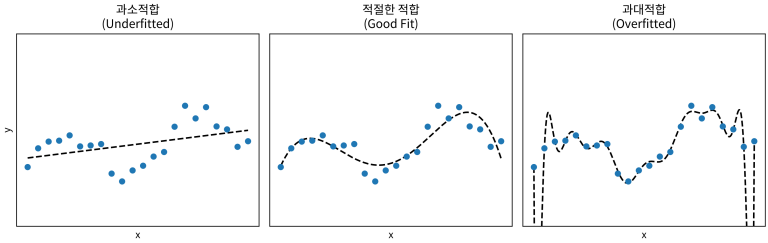
과대적합: 24차 다항식

그 반대 — “**외운** 모델”은 어떤 모습일까? 훈련 행 앞 25개에 고차 다항식을 맞춰봄:

```
Xs, ys = Xtr[:25], ytr[:25]
mu, sd = Xs.mean(), Xs.std()          # 표준화 (Week 4.1 스케일문제 )
for deg in [1, 24]:
    V = np.vander((Xs - mu) / sd, N=deg + 1)  # 설계행렬  [1, x, x^2, ...]
    coef, *_ = np.linalg.lstsq(V, ys, rcond=None)
    Vt = np.vander((Xte - mu) / sd, N=deg + 1)
    print(f"degree {deg}: train R2={r2(ys, V @ coef):.4f}  test R2={r2(yte, Vt @ coef):.4f}")
```

```
degree 1: train R2=0.8820 test R2=0.9368     degree 24: train R2=1.0000 test R2=-63495.2701
```

과대적합(overfitting)의 교훈



- 직선(과소적합)은 데이터의 굴곡을 놓치고, 16차 다항식(과대적합)은 **점과 점 사이의 잡음까지** 외워 양 끝에서 요동친다 — 실데이터로는 degree 1 vs 24 다항식 회귀에서 훈련 $R^2 = 1.0000$(완벽해 보임)인데 테스트 R^2는 **마이너스 6만**("평균만 예측"보다 몇만 배 나쁨).
- 25개 파라미터가 25개 점에 정확히 맞춰지는 순간, 다항식은 점 사이에 있던 **잡음 자체**를 외워버림.

핵심 교훈

"훈련 점수가 높다"는 사실은 잘 배웠다는 증거가 아니라, 외웠을 가능성을 알리는 신호일 수 있으며, 모델이 좋은지 나쁜지를 결정하는 것은 언제나 보지 않은 데이터의 점수다. (엄밀한 처리는 Week 6.)

수학과 Python 사전지식 리뷰 — 세 영역

이번 학기는 다음 세 영역의 수학을 **계속 재사용**한다 — 낯선 표기가 나올 때마다 이 절로 돌아와 확인할 것:

- **선형대수**: 벡터의 **내적** $w^\top x = \sum_j w_j x_j$, **행렬-벡터 곱**, 그리고 **그래디언트**(다변수 함수를 각 변수로 편미분해서 만든 벡터, $\nabla_w J$). 고유벡터/고유값은 Week 14(PCA)에서 다시.
- **미적분**: **연쇄법칙 (chain rule)** — 합성함수 $f(g(x))$ 의 미분이 $f'(g(x))\,g'(x)$ 라는 것. **Week 9(역전파)의 핵심 도구**가 바로 이것.
- **확률**: **조건부 확률**과 **베이즈 정리**

$$P(y \mid x) = \frac{P(x \mid y)\,P(y)}{P(x)}$$

Week 3에서 본격적으로 사용.

손으로 한 번: 내적(4)과 연쇄법칙(42)

표기가 낯설다면 **아주 작은 숫자로 직접 계산**해보는 것이 가장 빠르다.

- **내적**: $w = [2, -1, 3]$, $x = [1, 4, 2]$ 일 때:

$$w^\top x = (2)(1) + (-1)(4) + (3)(2) = 2 - 4 + 6 = 4$$

  각 자리끼리 곱해서 다 더하는 것뿐 — 이 계산이 이번 학기 **거의 모든 주차의 첫 줄**에 등장.

- **연쇄법칙**: $g(x) = 3x + 1$, $f(u) = u^2$, $x = 2$ 일 때 $f(g(x)) = (3x+1)^2$ 의 미분:

$$f'(g(x)) \cdot g'(x) = 2(3x+1) \cdot 3 = 2 \cdot 7 \cdot 3 = 42$$

  직접 전개($(9x^2 + 6x + 1$ 의 도함수 $18x + 6$, $x = 2$ 에서)해도 **똑같이 42** — 연쇄법칙은 "안쪽을 미분하고 바깥쪽을 미분해서 **곱한다**"는 지름길일 뿐, 다른 답을 주는 게 아니다.

두 가지를 기억하자

Week 9에서 이 연쇄법칙 계산을 **한 번이 아니라 수십 개의 층을 거쳐 반복**하게 된다 — 지금 이 작은 예제가 그 전체 과정의 **축소판**. 실전에서는 numpy/PyTorch로 훨씬 빠르게 같은 계산을 하지만, 처음 배울 때는 벡터 연산 뒤에 **숨은 for 루프를 직접 눈으로 보는 쪽**이 이해에 더 도움이 된다는 판단에서, 개념 코드는 순수 Python(리스트 + for 루프)으로 짠 경우가 많다.

행렬-벡터 곱: numpy로 한 줄

```python
import numpy as np

X = np.array([[1.0, 2.0], [3.0, 4.0]])  # 2x2 행렬
w = np.array([0.5, -0.5])               # 길이 2 벡터
print(X @ w)   # 행렬벡터- 곱: [-0.5, -0.5]
```

- `X @ w`(행렬곱)는 앞으로 매 장에 나오는 $w^\top x$ 형태의 계산을 **한 줄**로 표현 — for 루프로 직접 짜는 것보다 **빠르고 읽기 쉬움**.
- 이번 학기 수학의 전량은 **내적(4) + 연쇄법칙(42)** 이 두 계산의 "여러 번, 다른 형태로 반복"에 불과하다.

확인 문제: 파이프라인 원칙을 실제 상황에서 (a)(b)

- **(a)** 이탈 라벨이 붙은 10,000행에서 EDA 결과 “이탈”이 **2%뿐**. 이 사실이 파이프라인의 어느 단계에 무엇을 바꾸는가?
- **답**: **EDA의 발견이 평가 단계(지표 선택)를 바꿈** — 정확도는 “전부 비이탈”로 예측해도 98%라 이 문제 에선 무의미에 가까움. **PR-AUC**(Week 2.3) 같은 불균형 클래스 지표를 처음부터 염두에 두고, 이후 단계(클래스 가중, 샘플링 전략)를 계획.

- **(b)** 1,000행에서 age 결측이 훈련부 10개, 테스트부 3개. 평균으로 채운다 — **어느 부분**의 평균을 계산해야 하는가?
- **답**: **훈련 부분** — “훈련에서 fit, 테스트에 apply” 원칙. 전체 평균을 쓰면 테스트 정보가 훈련에 살짝 새는 형태.

확인 문제 (c)(d)

- **(c)** 어떤 모델의 train R^2 = **0.99**, test R^2 = **0.41**. 가장 가능성 높은 진단은? 24차 다항식 예제와 어떻게 같은 이야기인가?
- **답**: **과대적합** — 모델이 훈련 데이터의 규칙 이 아니라 **잡음을 외움**. 24차 예제(train R^2=1.0000, test R^2=−63495)가 이 현상의 **극단적 버전**. 두 경우 모두 "**훈련 점수가 높다**는 사실 자체"가 의심의 대상.

- **(d)** 새 데이터를 받아 실행 **전에**, 특징 벡터가 완전히 같은 행 두 개가 **서로 다른 라벨**을 갖고 있음을 발견. 가장 먼저 무엇을 해야 하는가?
- **답**: **데이터 문제를 먼저 의심** — Aha! 데이터셋 사고가 정확히 이 상황. `df.duplicated(keep=False)` 로 몇 개인지 파악한 뒤, 하나를 제거할지 라벨을 재확인할지를 **도메인 지식을 바탕으로** 결정 (알고리즘을 바꾸는 문제가 아님).

1.3 마무리

한 줄 요약

모델은 파이프라인 다섯 단계 중 하나일 뿐 — 그리고 모델의 상한을 가장 결정적으로 좌우하는 단계는 모델이 아니라, “데이터가 얼마나 깨끗하고 충분히 이해되었는가”다.

- 파이프라인: **수집** → **정제** → **EDA** → **모델링** → **평가** — 결함은 위에서 아래로 **증폭**.
- train/test는 **나누기 전에** 통계량을 계산하면 **누수** — “훈련에서 fit, 테스트에 apply”.
- 수학 전량은 **내적(4) + 연쇄법칙(42) + 베이즈 정리** — 낯선 표기가 나올 때마다 이 절로.

이번 주차 정리

1. **1.1 역사와 로드맵**
   - 역사는 직선이 아님 — **두 번의 AI 겨울**(1969 XOR 증명, 1986 역전파로 해소)과 2012 AlexNet.
   - 돌파구의 반복 패턴: **새 수학이 아니라 데이터+ 계산의 재조합**. Block A(Week 2∼7 고전 ML) → Block B(Week 9∼16 신경망→생성모델).
2. **1.2 문제 정식화와 세 갈래**
   - 알고리즘보다 먼저: x • y • h **로 정의**. "라벨이 있는가? 보상이 있는가?" 두 질문으로 **지도/비지도/강화**를 가름.
   - 회귀 vs 분류 = **같은 모델, 다른 출력**(항등 vs 시그모이드 링크) · 모든 알고리즘의 뼈대 = J **정의** → **최소화**.
3. **1.3 파이프라인과 사전지식**
   - 모델링은 5단계 중 하나 — 더티 데이터(결측·중복· 이상치)·EDA·**train/test 순서(누수 방지)**가 모델의 상한을 결정.
   - 수학 전량 = **내적 · 연쇄법칙 · 베이즈 정리**.

다음 주차 — Week 2. 회귀 모델: 선형회귀와 로지스틱회귀

이 책의 첫 번째 모델. 그 도입부에는 이미 19세기 수학자가 잡음 섞인 점들 사이를 "모두를 가장 잘 맞혀주는" 한 줄로 통과시켜야 할 문제가 등장한다 — 오늘 세운 "**손실함수를 정의하고 최소화한다**"는 틀과